

CropCare

Crop Disease Information Portal

Software Requirements Specification

Version 1.0 | IEEE-Style SRS Document

Project Title	CropCare - Crop Disease Information Portal
Document Type	Software Requirements Specification (SRS)
Category	Academic / College Software Engineering Project
Technology Stack	HTML5, CSS3, JavaScript, Browser Local Storage
Prepared For	Faculty Evaluation & Project Submission
Version	1.0
Date	August 2026

Table of Contents

1. Introduction	3
2. Existing System	4
3. Proposed System	5
4. Overall Description	6
5. System Requirements	8
6. Functional Requirements	9
7. Use Case Specifications	14
8. Non-Functional Requirements	19
9. User Interface Requirements	20
10. Data Requirements	22
11. System Architecture	23
12. System Workflow	24
13. Validation and Error Handling	25
14. Testing Requirements	26
15. Constraints	28
16. Future Scope	28
17. Expected Outcome	29
18. Conclusion	29

Page references correspond to the formatted PDF layout and may shift slightly with printer settings.

1. Introduction

1.1 Purpose

The purpose of this document is to provide a complete Software Requirements Specification (SRS) for **CropCare – Crop Disease Information Portal**, a frontend web application developed as a college software engineering project. This document describes the functional and non-functional requirements, system design considerations, use cases, and constraints of the system in sufficient detail to guide development, testing, evaluation, and future enhancement. It is intended to serve as a shared reference between the student developer(s) and evaluating faculty, ensuring that the delivered application matches the agreed scope.

1.2 Project Overview

CropCare is a modern, frontend-based information portal that helps students, farmers, agricultural enthusiasts, and the general public explore common crops and learn about the diseases that affect them. The platform organizes disease information – symptoms, causes, prevention, treatment, and severity – into a clean, searchable, and filterable interface. CropCare is built entirely using HTML5, CSS3, and JavaScript, with Browser Local Storage used to persist user preferences such as bookmarks and theme selection. No backend server, database, or authentication system is used in the current version.

1.3 Scope

The scope of CropCare (current version) covers the presentation and exploration of predefined crop and disease data through a responsive, client-side web application. In scope: crop browsing, crop and disease search, disease detail viewing, disease filtering, a client-side bookmarking system, general farming tips, a dark/light theme toggle, an FAQ section, and a client-validated contact/feedback form. Out of scope for the current version: user authentication, an admin dashboard, a backend server or database, payment processing, real-time or AI-based disease detection, weather API integration, and GPS/location-based services. These excluded items are discussed only under Future Scope (Section 16).

1.4 Intended Users

- **Students and Agricultural Enthusiasts** – browsing crop and disease information for learning purposes.
- **Small and Marginal Farmers** – looking up symptoms and prevention/treatment guidance for common crop diseases.
- **Faculty and Evaluators** – reviewing the project for academic assessment, viva, and demonstration.
- **General Public** – casual visitors interested in basic agricultural and crop-care knowledge.

1.5 Definitions and Abbreviations

Term	Definition
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
DOM	Document Object Model, the JavaScript-accessible structure of an HTML page
Local Storage	A browser-provided key-value storage mechanism that persists data on the client across page reloads
Client-Side	Logic and processing that runs entirely within the user's browser, without server involvement
Responsive Design	A design approach that adapts layout and content to different screen sizes and devices
LLM	Large Language Model – referenced only as an optional future enhancement

2. Existing System

2.1 Existing Approach

At present, individuals seeking information about crop diseases typically rely on scattered sources: general web search engines, government agricultural bulletins in PDF form, printed pamphlets distributed by agricultural departments, word-of-mouth advice from other farmers, or generic gardening blogs. Information is rarely organized by crop and disease in a consistent, structured, and searchable format designed specifically for quick client-side lookup.

2.2 Problems in Existing Approach

- Information is fragmented across many unrelated websites and documents, with no single organized reference point.
- Search engine results are inconsistent in quality, often mixing unrelated or unreliable content with useful advice.
- Government and extension bulletins are frequently long PDFs that are not easy to search, filter, or browse on a mobile device.
- There is no easy way to bookmark or revisit specific disease information for later reference.
- Most existing resources are not designed with a lightweight, mobile-friendly, offline-tolerant interface in mind.

2.3 Limitations

- Lack of a unified, crop-wise and disease-wise categorized information structure.
- Absence of quick search and filter tools tailored to crop diseases specifically.
- No lightweight personalization (such as saving frequently referenced diseases) in most public informational resources.
- Poor responsiveness and readability on smaller screens for many existing agricultural information sources.

3. Proposed System

3.1 Proposed Solution

CropCare proposes a single, organized, frontend-based web portal that consolidates crop and crop-disease information into a consistent format. Users can browse crops, search or filter diseases, view structured disease details (symptoms, causes, prevention, treatment, severity), bookmark diseases of interest using Browser Local Storage, and access general farming tips – all within a fast, responsive, and visually clean interface that works across devices without requiring installation, login, or a backend server.

CropCare focuses on doing one thing well: presenting organized, easy-to-find crop disease information through a clean and responsive client-side interface.

3.2 Objectives

- To provide a centralized, structured source of crop and crop-disease information.
- To enable fast search and filtering of crops and diseases by name, crop, disease type, and severity.
- To allow users to bookmark diseases for quick future reference using Local Storage.
- To present farming tips and agricultural best practices alongside disease information.
- To deliver a fully responsive, accessible, and visually consistent experience across desktop, tablet, and mobile devices.
- To demonstrate sound frontend software engineering practice suitable for academic evaluation.

3.3 Advantages

- Single, consistent, and organized interface for crop disease information.
- No installation, login, server, or database required to use the current version.
- Fast client-side search and filtering with instant results.
- Persistent bookmarks and theme preference via Local Storage, without user accounts.
- Lightweight and portable – runs in any modern browser.
- Clear, realistic architecture that can be extended with a backend in the future without a full redesign.

3.4 Major Features

- Home Page with introduction, hero section, and featured crops
- Crop Explorer with individual crop pages and related diseases
- Crop and Disease Search with dynamic, JavaScript-driven results
- Structured Disease Information pages (symptoms, causes, prevention, treatment, severity)
- Disease Filtering by crop, disease type, and severity
- Bookmark System backed by Browser Local Storage
- Farming Tips section covering soil, water, pest, and crop-care practices
- Light/Dark Mode toggle with saved preference
- FAQ section with accordion-style interaction
- Contact/Feedback form with client-side validation
- Fully Responsive Design for desktop, laptop, tablet, and mobile

4. Overall Description

4.1 Product Perspective

CropCare is a standalone, self-contained frontend web application. It is not a component of any larger existing system and does not currently depend on any external server or third-party backend service. It is designed as an independent academic project that can later be integrated with a backend API and database without requiring a redesign of the user interface layer.

4.2 Product Functions

- Display featured crops and general agricultural information on the Home Page.
- Allow browsing of crops and their associated diseases through the Crop Explorer.
- Provide dynamic search for crops and diseases by name.
- Display detailed, structured information for each disease.
- Allow filtering of diseases by crop, disease type, and severity level.
- Allow users to bookmark and remove bookmarked diseases, persisted via Local Storage.
- Present farming tips categorized by topic (soil, water, pest, crop protection).
- Allow switching between light and dark themes, with the preference remembered.
- Present frequently asked questions in an interactive accordion format.
- Allow users to submit a contact/feedback message with client-side validation.

4.3 User Characteristics

User Type	Technical Level	Primary Goal
Student / Learner	Basic to Moderate	Learn about crops and common diseases for study purposes
Farmer / Grower	Basic	Identify a disease and find prevention/treatment guidance quickly
Faculty / Evaluator	Moderate to Advanced	Assess project completeness, design quality, and requirement coverage
General Visitor	Basic	Casually explore agricultural information and farming tips

4.4 Operating Environment

- Runs entirely within a modern web browser on desktop, laptop, tablet, or mobile devices.
- No server-side runtime is required for the current version; the application can be served as static files.
- Requires JavaScript to be enabled in the browser for dynamic features (search, filtering, bookmarks, dark mode, accordion, form validation).
- Requires browser support for the Local Storage Web API.

4.5 Design Considerations

- Mobile-first, responsive layout using CSS Flexbox/Grid principles.
- Consistent agriculture-inspired visual identity (green color palette, clean typography).
- Modular, reusable JavaScript functions for search, filtering, bookmarking, and validation.
- Predefined crop and disease data structured as JavaScript objects/JSON for maintainability.
- Graceful handling of empty results, invalid input, and missing images.

4.6 Assumptions and Dependencies

- Users access the application using a modern, up-to-date web browser.
- Crop and disease data is predefined and bundled with the application (no live external data source).
- The device used has Local Storage available and not disabled by browser privacy settings.
- The current version does not depend on any external server, API, or database.
- Internet connectivity, if any, is only required to initially load the static files; no live network calls are made for core features.

5. System Requirements

5.1 Hardware Requirements

Component	Minimum Requirement
Processor	Any modern dual-core processor (1.5 GHz or higher)
RAM	2 GB minimum (4 GB recommended)
Storage	Negligible – static assets typically under 50 MB
Display	Any screen size from mobile (320px width) to desktop
Input Device	Touchscreen, mouse, or keyboard/trackpad

5.2 Software Requirements

Component	Requirement
Operating System	Any OS supporting a modern web browser (Windows, macOS, Linux, Android, iOS)
Browser	Modern browser with HTML5, CSS3, and ES6 JavaScript support
Development Tools	Code editor (e.g., VS Code), Git for version control
Hosting (optional)	Any static file host (e.g., GitHub Pages, Netlify) since no backend is required

5.3 Technology Stack

Layer	Technology
Structure	HTML5
Styling	CSS3 (Flexbox, Grid, Media Queries)
Behavior / Logic	JavaScript (ES6+)
Data Persistence (Client)	Browser Local Storage API
Data Source (Current)	Predefined JavaScript/JSON-style objects bundled with the application

5.4 Browser Requirements

CropCare is designed to run on current versions of major browsers, including Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari. Browsers must support HTML5, CSS3, ES6 JavaScript, and the Web Storage API (Local Storage). JavaScript must be enabled for search, filtering, bookmarking, dark mode, FAQ accordion, and form validation features to function correctly.

6. Functional Requirements

This section describes each functional requirement of CropCare with its purpose, description, inputs, processing behaviour, outputs, expected result, and applicable exception handling.

FR-01 Home Page

- Purpose:** Introduce the platform and provide quick access to key sections.
- Description:** The Home Page displays a hero section with a brief project introduction, a set of featured crops, a quick-access search bar, and basic agricultural information/highlights.
- Inputs:** None required; optional quick-search text entered by the user.
- Processing:** Page renders static hero content and dynamically populates the featured crop cards from the crop data set on load.
- Outputs:** Rendered hero section, featured crop cards, and navigation links to Crop Explorer, Farming Tips, FAQ, and Contact.
- Expected Result:** User understands the purpose of the site within seconds and can navigate to any major section in one click.
- Exception Handling:** If featured crop data fails to load, a friendly placeholder message is shown instead of a blank section.

FR-02 Crop Explorer

- Purpose:** Allow users to browse all available crops and view crop-specific information.
- Description:** Displays a grid/list of crops, each with an image, name, and short description. Selecting a crop opens its individual page listing related diseases.
- Inputs:** User click/tap on a crop card.
- Processing:** JavaScript reads the crop data set, renders crop cards, and on selection filters the disease data set for diseases matching that crop.
- Outputs:** Crop detail view showing crop description and a list of related diseases.
- Expected Result:** User can identify a crop and immediately see which diseases are associated with it.
- Exception Handling:** If a crop has no recorded diseases, a message such as "No diseases recorded for this crop yet" is displayed.

FR-03 Crop Search

- Purpose:** Let users quickly locate a crop by typing its name.
- Description:** A search input filters the crop list in real time as the user types.
- Inputs:** Text string entered by the user.
- Processing:** JavaScript performs a case-insensitive substring match against crop names and re-renders the matching results dynamically.
- Outputs:** Filtered list of matching crop cards.
- Expected Result:** Matching crops appear instantly without a page reload.
- Exception Handling:** If no crop matches the query, a clear "No crops found" message is displayed instead of an empty screen.

FR-04 Disease Search

- Purpose:** Let users quickly locate a disease by typing its name.
- Description:** A dedicated search field filters the disease data set in real time based on disease name.
- Inputs:** Text string entered by the user.
- Processing:** JavaScript performs a case-insensitive substring match against disease names and updates the result list dynamically.
- Outputs:** Filtered list of matching disease cards linking to their detail pages.
- Expected Result:** User quickly finds the relevant disease entry.
- Exception Handling:** Empty query returns the full disease list; unmatched query shows a "No diseases found" message.

FR-05 Disease Information

- Purpose:** Provide complete, structured information about a specific disease.
- Description:** For a selected disease, display its name, affected crop, image, description, symptoms, causes, prevention methods, treatment information, and severity level.
- Inputs:** Disease identifier passed from a card selection, search result, or filtered list.
- Processing:** JavaScript looks up the disease record by ID from the data set and renders all associated fields into the detail layout.
- Outputs:** Fully populated disease detail page.
- Expected Result:** User gains a complete understanding of the disease and applicable prevention/treatment steps.
- Exception Handling:** If an image fails to load, a placeholder graphic is shown; if a field is missing, it is omitted gracefully rather than showing broken text.

FR-06 Disease Filtering

- Purpose:** Allow users to narrow down the disease list using specific criteria.
- Description:** Users can filter diseases by crop, disease type, and severity level, individually or in combination.
- Inputs:** Selected filter option(s) from dropdowns/checkboxes.
- Processing:** JavaScript applies the selected filter criteria against the disease data set and re-renders the result list.
- Outputs:** Filtered disease list matching the selected criteria.
- Expected Result:** User views only the diseases relevant to their selected filters.
- Exception Handling:** If a filter combination returns no results, a message such as "No diseases match the selected filters" is shown along with an option to clear filters.

FR-07 Bookmark System

- Purpose:** Allow users to save diseases of interest for quick future reference.
- Description:** Users can bookmark and remove bookmarks for any disease; bookmarks are stored in Browser Local Storage and remain available after page refresh.
- Inputs:** Bookmark/remove-bookmark button click on a disease card or detail page.
- Processing:** JavaScript adds or removes the disease ID from a Local Storage array/object and updates the bookmark icon state.
- Outputs:** Updated bookmark indicator and an accessible "My Bookmarks" list.
- Expected Result:** Bookmarked diseases persist across sessions until explicitly removed by the user.
- Exception Handling:** If Local Storage is unavailable or full, the user is notified that bookmarks cannot be saved on this device/browser.

FR-08 Farming Tips

- Purpose:** Provide general agricultural best-practice guidance beyond disease-specific data.
- Description:** Displays categorized tips covering soil care, water management, pest prevention, and general crop-care practices.
- Inputs:** Optional category selection by the user.
- Processing:** JavaScript renders tip entries from a predefined tips data set, grouped by category.
- Outputs:** List/grid of farming tip cards.
- Expected Result:** User gains general crop-care knowledge alongside disease-specific information.
- Exception Handling:** Not applicable beyond standard empty-category handling.

FR-09 Dark Mode

- Purpose:** Improve usability and comfort in different lighting conditions.
- Description:** A toggle switch lets users switch between light and dark themes; the chosen theme is stored in Local Storage and re-applied on future visits.
- Inputs:** Toggle button click.
- Processing:** JavaScript toggles a theme class on the document root and saves the preference key to Local Storage; on page load, the saved preference is read and applied.
- Outputs:** Updated colour theme across the entire interface.
- Expected Result:** Theme preference persists across sessions without requiring login.
- Exception Handling:** If no preference is stored, the system defaults to light mode.

FR-10 FAQ

- Purpose:** Answer common questions about agriculture and the platform itself.
- Description:** Displays a list of frequently asked questions in an interactive accordion; clicking a question expands/collapses its answer.
- Inputs:** Click/tap on a question header.
- Processing:** JavaScript toggles the visibility (expanded/collapsed state) of the corresponding answer panel.
- Outputs:** Expanded answer text displayed beneath the selected question.
- Expected Result:** User can scan questions quickly and open only the answers relevant to them.
- Exception Handling:** Not applicable; purely presentational interaction.

FR-11 Contact / Feedback

- Purpose:** Allow users to send feedback or queries related to the platform.
- Description:** A form collects Name, Email, and Message fields with JavaScript-based client-side validation before displaying a success/error message.
- Inputs:** Name (text), Email (text), Message (text).
- Processing:** JavaScript validates that required fields are non-empty and that the email field matches a valid email pattern before accepting submission.
- Outputs:** Success confirmation message on valid submission, or field-specific error messages on invalid input.
- Expected Result:** User receives immediate, clear feedback about whether their submission was accepted.
- Exception Handling:** Empty required fields, invalid email format, and overly short messages are rejected with specific inline error messages. As this is a frontend-only project, submitted data is confirmed on-screen and not sent to a live backend in the current version.

FR-12 Responsive User Interface

- Purpose:** Ensure a consistent, usable experience across all device sizes.
- Description:** All pages and components adapt their layout using CSS media queries and flexible grid/flex layouts for desktop, laptop, tablet, and mobile screens.
- Inputs:** Device/viewport width detected by the browser.
- Processing:** CSS media queries adjust grid columns, font sizes, spacing, and navigation layout (e.g., collapsing into a mobile menu) based on breakpoints.
- Outputs:** Layout appropriately re-arranged for the current screen size.
- Expected Result:** All features remain fully usable and readable regardless of device.
- Exception Handling:** Very small or unusual viewport sizes gracefully fall back to a single-column stacked layout.

7. Use Case Specifications

This section details each use case supported by CropCare, including actors, preconditions, flows, and postconditions.

UC-01 Browse Crops

Purpose: Allow the user to explore the list of available crops.

Primary Actor: Any Visitor

Preconditions: The application has loaded successfully with crop data available.

Trigger: User navigates to the Crop Explorer page.

Main Flow:

1. User selects “Crop Explorer” from the navigation bar.
2. System renders all crop cards with image, name, and short description.
3. User scrolls/browses through the crop list.
4. User selects a crop to view related diseases (leads to UC-04).

Alternative Flow:

User may use Crop Search (UC-02) instead of scrolling manually.

Exception Flow:

If crop data fails to render, a fallback message “Unable to load crops” is displayed.

Postconditions: User has viewed the crop list and optionally selected a crop.

UC-02 Search Crop

Purpose: Allow the user to quickly locate a specific crop by name.

Primary Actor: Any Visitor

Preconditions: Crop Explorer page is loaded.

Trigger: User types text into the crop search field.

Main Flow:

1. User enters a crop name (or partial name) into the search box.
2. System filters the crop list in real time using a case-insensitive match.
3. Matching crop cards are displayed.
4. User selects a crop from the results.

Alternative Flow:

User clears the search field to return to the full crop list.

Exception Flow:

If no crop matches, system displays “No crops found for your search.”

Postconditions: A filtered or full crop list is displayed based on the query.

UC-03 Search Disease

Purpose: Allow the user to quickly locate a specific disease by name.

Primary Actor: Any Visitor

Preconditions: Disease Search interface is loaded.

Trigger: User types text into the disease search field.

Main Flow:

1. User enters a disease name (or partial name).
2. System filters the disease data set in real time.
3. Matching disease cards are displayed with crop and severity indicators.
4. User selects a disease to view full details (leads to UC-04).

Alternative Flow:

User combines search with filters (UC-05) for more precise results.

Exception Flow:

If no disease matches, system displays "No diseases found for your search."

Postconditions: A filtered or full disease list is displayed based on the query.

UC-04 View Disease Details

Purpose: Present complete structured information for a selected disease.

Primary Actor: Any Visitor

Preconditions: A valid disease record exists and is selected.

Trigger: User clicks a disease card from Explorer, Search, or Filter results.

Main Flow:

1. User selects a disease.
2. System retrieves the disease record by ID.
3. System renders name, affected crop, image, description, symptoms, causes, prevention, treatment, and severity.
4. User optionally bookmarks the disease (leads to UC-06).

Alternative Flow:

User navigates back to Explorer or Search results without bookmarking.

Exception Flow:

If the disease image fails to load, a placeholder image is displayed; missing optional fields are omitted gracefully.

Postconditions: Complete disease information has been presented to the user.

UC-05 Filter Diseases

Purpose: Allow the user to narrow the disease list by crop, disease type, and/or severity.

Primary Actor: Any Visitor

Preconditions: Disease listing page is loaded with filter controls visible.

Trigger: User selects one or more filter options.

Main Flow:

1. User opens filter controls (crop, disease type, severity).
2. User selects one or more filter values.
3. System applies all active filters simultaneously and re-renders the results.
4. User views the filtered disease list.

Alternative Flow:

User clears one or all filters to widen results.

Exception Flow:

If the combined filters return zero results, system displays "No diseases match the selected filters" with a "Clear Filters" option.

Postconditions: Disease list reflects the currently active filter criteria.

UC-06 Bookmark Disease

Purpose: Allow the user to save a disease for quick future reference.

Primary Actor: Any Visitor

Preconditions: A disease card or detail page is being viewed; Local Storage is available.

Trigger: User clicks the bookmark icon/button.

Main Flow:

1. User clicks the bookmark button on a disease.
2. System adds the disease ID to the bookmarks list in Local Storage.
3. Bookmark icon updates to a "saved" state.
4. Disease becomes visible in the "My Bookmarks" view.

Alternative Flow:

None.

Exception Flow:

If Local Storage is unavailable/disabled, system shows a notice that bookmarking is not supported on this browser/device.

Postconditions: The disease is persisted as a bookmark and remains available after page refresh.

UC-07 Remove Bookmark

Purpose: Allow the user to remove a previously bookmarked disease.

Primary Actor: Any Visitor

Preconditions: The disease is currently bookmarked.

Trigger: User clicks the remove/unbookmark icon.

Main Flow:

1. User clicks the "remove bookmark" button on a saved disease.
2. System removes the disease ID from the Local Storage bookmarks list.
3. Bookmark icon reverts to the default (unsaved) state.
4. Disease disappears from the "My Bookmarks" view.

Alternative Flow:

None.

Exception Flow:

If the Local Storage update fails, system retains the current state and notifies the user.

Postconditions: The disease is no longer listed under bookmarks.

UC-08 View Farming Tips

Purpose: Allow the user to browse general agricultural best-practice tips.

Primary Actor: Any Visitor

Preconditions: Farming Tips section/page is available.

Trigger: User navigates to the Farming Tips section.

Main Flow:

1. User opens the Farming Tips section.
2. System renders tip cards grouped by category (soil, water, pest, crop protection).
3. User browses or selects a category to narrow the view.

Alternative Flow:

User browses all categories without filtering.

Exception Flow:

If a category has no tips, a placeholder message is shown.

Postconditions: User has viewed general farming guidance.

UC-09 Toggle Dark Mode

Purpose: Allow the user to switch between light and dark visual themes.

Primary Actor: Any Visitor

Preconditions: Application has loaded.

Trigger: User clicks the theme toggle control.

Main Flow:

1. User clicks the dark/light mode toggle.
2. System applies the corresponding theme class to the page.
3. System saves the selected theme to Local Storage.
4. On future visits, the saved theme is automatically applied.

Alternative Flow:

None.

Exception Flow:

If no saved preference exists, system defaults to light mode.

Postconditions: The selected theme is applied and remembered for future sessions.

UC-10 Use FAQ

Purpose: Allow the user to view answers to common questions.

Primary Actor: Any Visitor

Preconditions: FAQ section is loaded.

Trigger: User clicks a question header.

Main Flow:

1. User clicks on a collapsed FAQ question.
2. System expands the accordion panel to reveal the answer.
3. User may click again to collapse it, or click another question.

Alternative Flow:

Multiple panels may be open simultaneously, depending on implementation choice.

Exception Flow:

Not applicable; purely presentational.

Postconditions: User has viewed the answer(s) to selected question(s).

UC-11 Submit Contact/Feedback Form

Purpose: Allow the user to send feedback, questions, or comments about the platform.

Primary Actor: Any Visitor

Preconditions: Contact/Feedback page is loaded.

Trigger: User clicks the "Submit" button after filling the form.

Main Flow:

1. User enters Name, Email, and Message.
2. User clicks Submit.
3. System validates all fields on the client side.
4. If valid, system displays a success confirmation message.

Alternative Flow:

User corrects a field after seeing a validation error and resubmits.

Exception Flow:

Empty required fields, invalid email format, or too-short messages trigger field-specific error messages and block submission.

Postconditions: A confirmation message is shown to the user on successful validation; the form may be cleared for a fresh entry.

Special Requirements: As CropCare is currently a frontend-only project, submitted data is confirmed on-screen and is not persisted to a live backend in this version.

8. Non-Functional Requirements

8.1 Usability

The interface shall be intuitive enough for a first-time visitor to locate crop and disease information without instructions, using clear labeling, consistent navigation, and visually distinct interactive elements.

8.2 Performance

Search, filtering, and bookmarking actions shall reflect visibly on screen within a fraction of a second, since all data is held client-side in memory and no network round-trip is required for these operations.

8.3 Responsiveness

All pages shall render correctly and remain fully usable across common breakpoints, including mobile (approx. 320–480px), tablet (approx. 481–1024px), and desktop (1025px and above) viewport widths.

8.4 Accessibility

The interface shall use sufficient colour contrast (including in dark mode), semantic HTML elements, descriptive alt text for images, and keyboard-navigable interactive controls such as the search box, filters, accordion, and form fields.

8.5 Compatibility

The application shall function correctly on the latest two major versions of Chrome, Firefox, Edge, and Safari, on both desktop and mobile operating systems.

8.6 Reliability

Core browsing features (Home, Crop Explorer, Disease Information, Farming Tips, FAQ) shall remain functional even if Local Storage is unavailable; only bookmark persistence and saved theme preference depend on Local Storage.

8.7 Maintainability

Crop and disease data shall be stored as structured JavaScript objects/JSON, separated from presentation logic, so that new crops or diseases can be added by extending the data set without modifying core application logic.

8.8 Security

As a frontend-only application with no backend or user accounts in the current version, security considerations are limited to client-side input sanitisation on the contact form to avoid malformed data display, and safe handling of Local Storage data.

8.9 Scalability

The data-driven architecture (crops and diseases as structured objects) shall allow the underlying data set to grow considerably without requiring structural changes to the rendering, search, or filter logic.

9. User Interface Requirements

UI Element	Requirement
Home Page	Hero banner, short introduction, featured crop cards, quick-access search bar
Navigation Bar	Persistent links to Home, Crop Explorer, Farming Tips, FAQ, Contact, Bookmarks, and Dark Mode toggle; collapses into a mobile menu on small screens
Hero Section	Prominent heading, short tagline, and a call-to-action button leading to Crop Explorer
Search Interface	Single input field with live filtering feedback and a clear/reset control
Crop Cards	Image, crop name, one-line description, and a link/button to view details
Disease Cards	Image, disease name, affected crop, severity badge, and bookmark icon
Disease Details Page	Structured layout showing image, description, symptoms, causes, prevention, treatment, and severity, with a bookmark control
Filter Controls	Dropdowns/checkboxes for crop, disease type, and severity, with an active-filter summary and clear option
Bookmark Interface	A dedicated "My Bookmarks" view listing saved diseases with quick remove controls
Farming Tips	Category-grouped tip cards with concise, scannable text
FAQ Accordion	Collapsible question/answer panels with a clear expand/collapse indicator
Contact Form	Name, Email, and Message fields with inline validation messages and a Submit button
Dark Mode	Toggle switch, typically in the navigation bar, with a smooth colour transition
Footer	Basic project information, quick links, and attribution
Mobile Responsive Layout	Single-column stacking, larger touch targets, and a collapsible navigation menu

10. Data Requirements

CropCare's current version uses predefined, structured client-side data rather than a live database. Data is represented as JavaScript objects styled as JSON, which keeps the structure predictable and easy for search, filter, and rendering functions to consume.

10.1 Crop Data

Field	Description
Crop ID	Unique identifier for the crop
Crop Name	Display name of the crop
Description	Short overview of the crop
Image	Reference to the crop's image asset
Season	Typical growing season for the crop
Related Diseases	List of Disease IDs associated with the crop

10.2 Disease Data

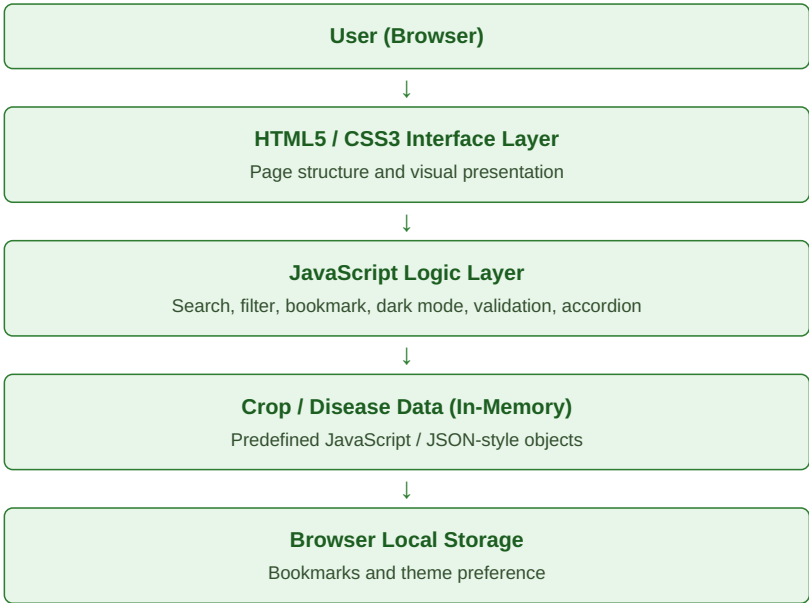
Field	Description
Disease ID	Unique identifier for the disease
Disease Name	Display name of the disease
Crop	Crop ID the disease is associated with
Disease Type	Category such as fungal, bacterial, viral, or nutritional
Description	General overview of the disease
Symptoms	Observable signs of the disease
Causes	Underlying causes/conditions leading to the disease
Prevention	Recommended preventive measures
Treatment	Recommended treatment guidance
Severity	Severity classification, e.g., Low, Medium, High
Image	Reference to the disease's image asset

10.3 Use of JavaScript Data and Local Storage

Crop and disease records are held in JavaScript arrays of objects (JSON-style) that are bundled with the application and loaded into memory when the page loads. All search, filter, and detail-view functions operate on these in-memory arrays. Browser Local Storage is used only for data that must persist between visits and is specific to the individual user's browser: the list of bookmarked Disease IDs, and the selected theme (light/dark) preference. This keeps the core content centrally maintainable while still offering lightweight personalization without accounts or a backend.

11. System Architecture

CropCare follows a simple, layered client-side architecture, appropriate for a frontend-only application.



11.1 Future Backend Integration Path

Although the current version has no backend, the architecture is intentionally layered so that the JavaScript Logic Layer could later call a REST API instead of reading from in-memory data, without requiring any change to the HTML/CSS interface layer. In such a future version, crop and disease data could be served from a database through an API layer, while Local Storage could continue to be used for lightweight client-side personalization such as bookmarks, or be replaced by server-side user accounts.

12. System Workflow

12.1 Primary Navigation Workflow



12.2 Search Workflow



12.3 Disease Filtering Workflow



12.4 Bookmark Workflow



12.5 FAQ Workflow



12.6 Contact Workflow



12.7 Dark Mode Workflow



13. Validation and Error Handling

Scenario	System Behaviour
Empty Search	Displays the full unfiltered crop/disease list
Invalid Search (special characters, etc.)	Treated as plain text; safely produces a "No results" state if unmatched
No Search Results	Displays a clear "No crops/diseases found" message with a suggestion to clear the search
Invalid Form Fields (empty Name/Message)	Inline error message shown below the specific field; submission blocked
Invalid Email	Inline error message indicating an invalid email format; submission blocked until corrected
Local Storage Failure/Unavailable	Bookmark and dark-mode preference features degrade gracefully with a notice; core browsing remains functional
Missing Data Fields	Optional missing fields are omitted from display rather than showing blank or broken text
Broken Image	A placeholder image/icon is displayed in place of the missing crop/disease image

14. Testing Requirements

Testing for CropCare covers functional correctness, UI consistency, responsiveness, browser compatibility, and client-side data handling. The following testing types shall be performed:

- Functional Testing – verifying each feature behaves as specified in Section 6.
- UI Testing – verifying visual consistency, layout, and theme correctness.
- Responsive Testing – verifying layout correctness across mobile, tablet, laptop, and desktop widths.
- Browser Compatibility Testing – verifying consistent behaviour across Chrome, Firefox, Edge, and Safari.
- Search Testing – verifying accurate, case-insensitive, real-time search results for crops and diseases.
- Filter Testing – verifying correct results for single and combined filter criteria.
- Bookmark Testing – verifying add, remove, and persistence of bookmarks across page reloads.
- Local Storage Testing – verifying correct read/write behaviour and graceful handling when storage is unavailable.
- Form Validation Testing – verifying correct acceptance/rejection of valid and invalid form inputs.
- Usability Testing – verifying ease of navigation and clarity of information for first-time users.

14.1 Sample Test Cases

Test Case ID	Feature	Input	Expected Result	Actual Result	Status
TC-01	Crop Search	"Wheat"	Displays crop card(s) matching "Wheat"	As expected	Pass
TC-02	Disease Search	"Blight"	Displays disease card(s) containing "Blight"	As expected	Pass
TC-03	Disease Search	"xyzxyz" (no match)	Displays "No diseases found" message	As expected	Pass
TC-04	Disease Filtering	Severity = High	Displays only diseases marked High severity	As expected	Pass
TC-05	Bookmark System	Click bookmark on a disease	Disease appears in My Bookmarks and persists after refresh	As expected	Pass
TC-06	Remove Bookmark	Click remove on a bookmarked disease	Disease removed from My Bookmarks list	As expected	Pass
TC-07	Dark Mode Toggle	Click theme toggle	Theme switches and is remembered on next visit	As expected	Pass
TC-08	Contact Form	Invalid email "abc@"	Inline error: invalid email format; submission blocked	As expected	Pass
TC-09	Contact Form	All fields valid	Success confirmation message displayed	As expected	Pass
TC-10	Responsive Layout	Viewport width 375px	Layout stacks into single column; nav collapses to menu	As expected	Pass
TC-11	FAQ Accordion	Click a question	Answer panel expands; clicking again collapses it	As expected	Pass

Test cases are illustrative; the full test suite should be executed against the actual implementation prior to submission.

15. Constraints

- The current version is a frontend-only implementation with no backend server or live database.
- Crop and disease data is predefined and bundled with the application rather than sourced dynamically.
- The project has no backend/database in the current version; all persistence is limited to Browser Local Storage.
- Bookmarks and theme preference depend entirely on Browser Local Storage and are specific to a single browser/device.
- Development time is limited by the academic project timeline available to the student team.
- Agricultural information presented is static/predefined and is not sourced from a live, continuously updated agricultural database.

16. Future Scope

The following enhancements are **not implemented in the current version** and are listed only as realistic directions for future development:

- Backend database for centrally managed, continuously updated crop and disease records Future Scope
- User accounts and authentication for personalized, cross-device bookmarks Future Scope
- Admin content management system for adding/editing crops and diseases Future Scope
- AI-based leaf image disease detection Future Scope
- Weather-based disease alert notifications Future Scope
- Multilingual support for regional accessibility Future Scope
- Location-based agricultural services using GPS Future Scope
- LLM-based Farming Assistant, offering AI-generated educational responses to general crop-care questions Future Scope

None of the items above are part of the current implemented feature set; they are documented solely for future planning purposes.

17. Expected Outcome

Upon completion, CropCare is expected to deliver a fully functional, responsive, frontend-only web portal that allows users to browse crops, search and filter disease information, view structured disease details, bookmark diseases of interest, read general farming tips, switch between light and dark themes, consult an FAQ section, and submit validated contact/feedback messages. The application is expected to demonstrate solid frontend engineering practice – clean HTML structure, organized CSS styling, and modular JavaScript logic – suitable for academic evaluation, live demonstration, and viva discussion, while remaining realistically achievable within a student development timeline.

18. Conclusion

CropCare addresses a genuine, practical problem – the lack of a single, organized, easy-to-search source of crop disease information – through a focused, achievable, frontend-only solution built with HTML5, CSS3, JavaScript, and Browser Local Storage. By concentrating on well-structured data, clear search and filtering, lightweight personalization through bookmarks, and a fully responsive interface, CropCare demonstrates practical frontend software development skills appropriate for a college-level software engineering project. The architecture is deliberately layered so that future enhancements – including a backend, database, authentication, and AI-assisted features – can be added later without redesigning the core user interface, making CropCare both a complete standalone deliverable today and a reasonable foundation for future growth.